
Data Structures

BS (CS) _Fall_2025

Lab_02 Task



Learning Objectives:

1. Class Templates with unit testing (Gtest)

Class Templates in C++

Task 0: Simple Generic Holder

Create a template class `Holder<T>` that stores a single item of any type.

Data Members:

- **T item** : the stored value.

Functions:

- **Constructor**: initialize the item.
- **setItem(T value)**: set the stored item.
- **getItem()**: return the stored item.
- **displayType()**: prints the data type of the stored item using `typeid(item).name()` from `typeinfo` header.

Test Cases:

- Create `Holder<int>`, `Holder<double>`, and `Holder<string>`.
- Set and get values for each.
- Print stored type and value.

Task 1: Generic Seating Arrangement

You are required to design a class template `Seating<T>` that manages a 2D seating arrangement (like an exam hall or a bus).

The `Seating<T>` class should represent seats arranged in rows and columns.

- Each seat can hold a value of type `T`.
- `T` could be a student name (`string`), a student ID (`int`), or simply an occupied/free status (`bool`).
- The class should provide functions to assign a seat, vacate a seat, check vacancy, and print the seating chart.

Additionally, you will implement a specialized version of this class for `bool`.

- In this case, each seat only stores whether it is occupied (`true`) or vacant (`false`).
- The `print()` function should show `O` for occupied seats and `-` for vacant seats.

Member Functions:

Constructor

- Takes number of rows and columns, allocates a 2D array of type T.

assignSeat(row, col, value)

- Assign a value of type T to a seat (e.g., "Ali", 101, or true).

vacateSeat(row, col)

- Empties a seat (removes value or sets false for bool).

isVacant(row, col)

- Returns whether a seat is empty.

printSeating()

- Prints the seating chart in a clean grid format.
- For bool specialization: show O for occupied and - for empty.

Test Case Descriptions for Seating<T>

- **Assign and Retrieve String Seat**
 - Assign "Ali" to (0,0) in Seating<string>.
 - Verify that the seat contains "Ali".
- **Vacate String Seat**
 - Assign "Sara" to (0,1), then vacate it.
 - Verify that (0,1) is vacant afterwards.
- **Assign and Retrieve Integer Seat**
 - Assign 101 to (1,0) in Seating<int>.
 - Verify that (1,0) contains 101.
- **Boolean Specialization , Occupied/Vacant**
 - In Seating<bool>, assign (0,0)=true and (0,1)=false.
 - Verify that (0,0) is occupied and (0,1) is vacant.

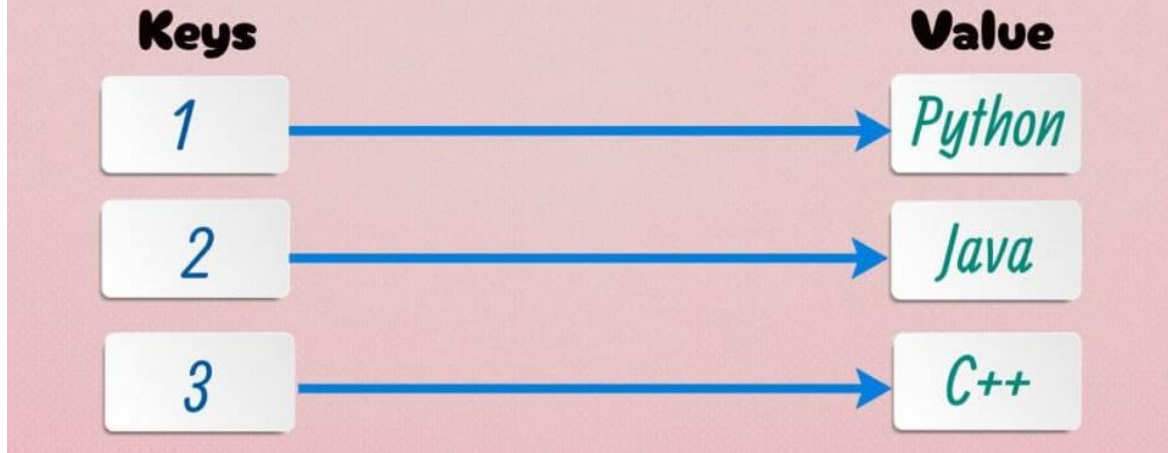
Task 2:

Template Class Dictionary:

You are designing a generic Dictionary class similar to Python's dict that stores key–value pairs of any type.

Dictionary in Python

Dict = {1: "Python", 2: "Java", 3: "C++"}



The dictionary should be flexible enough to:

- Work with any combination of key and value types.
- Allow capacity to be set at runtime when the object is created.
- Provide default types for convenience.
- Have a special behavior when keys are `std::string` → make it case-insensitive by automatically converting keys to lowercase and performing searches/removals ignoring case.

The class will have Three template parameters:

- K for the key type (default: `int`)
- V for the value type (default: `string`)

Data Members (Private)

- **K* keys;** :- dynamic array storing keys.
- **V* values;** :- dynamic array storing corresponding values.
- **int count;** :- current number of elements.
- **int currentSize;** :- current allocated size of arrays (starts small, doubles when needed).

Member Functions (Public)

Constructor

- Initializes `count = 0`, `currentSize = 2`.
- Allocates `keys = new K[currentSize]` and `values = new V[currentSize]`.

Destructor

- Deletes dynamically allocated arrays.

isEmpty()

- Returns true if count == 0.

insert(K key, V value)

- If the key already exists, do nothing.
- If count == currentSize, reallocate arrays with double the size.
- Insert the new key–value pair.

search(K key)

- Returns true if key exists, otherwise false.

remove(K key)

- If found, remove the key–value pair and shift the rest of the elements.

getValue(K key)

- Returns the value if key exists, otherwise throws an exception.

print()

- Prints all key–value pairs.

Test cases :

1. General Dictionary Tests (Default Parameters)(K = int, V = std::string)

- Create Dictionary<>.
- Insert 3+ key value pairs:
 - Example: 1 : "Ali", 2 : "Sara", 3 : "John". Insert duplicate key (e.g., 2 : "Sara") :- should not add.
- Search existing key (2) :- returns true.
- Search non-existing key (10) :- returns false.
- Get value of existing key (2) :- returns "Sara".
- Remove key (2) :- confirm it's gone.
- Print dictionary when empty and when not empty.

2. Auto resizing Tests : (K = int, V = double)

- Create Dictionary<int, double>.

- Insert more elements than initial size (e.g., > 2) with values like 101 : 90.5, 102 : 85.3, etc.
- Verify resizing happens and all inserted elements remain accessible.

3. String Key Specialization Tests (Case Insensitive)(K = std::string, V = int)

- Create Dictionary<std::string, int>.
- Insert "Apple" : 5, "Banana" : 10.
- Search "APPLE", "apple", "ApPIE" : all return true.
- Remove "APPLE" , confirm "apple" is removed.

4. Different Data Type Combination Tests

i) Double keys & char values

- Create Dictionary<double, char>.
- Insert 1.1 : 'A', 2.2 : 'B'.
- Search 2.2 , returns true.
- Remove 1.1 , confirm removal.

ii) Char keys & bool values

- Create Dictionary<char, bool>.
- Insert 'x' : true, 'y' : false.
- Search 'y' , returns true.
- Get value 'x' , returns true.